Figure 4.12 Hierarchical processing of agents coordinated through a crew manager

Adding this manager is a relatively simple process. Listing 4.20 shows the additional code changes to a new file that uses the coding crew in a hierarchical method. Aside from importing a class for connecting to OpenAI from LangChain, the other addition is adding this class as the crew manger, `manager_llm`.

Listing 4.20 `crewai_hierarchy.py` (crew manager sections)

```
from langchain_openai import ChatOpenAI      #1

crew = Crew(
    agents=[senior_engineer_agent,
            qa_engineer_agent,
            chief_qa_engineer_agent],
    tasks=[code_task, qa_task, evaluate_task],
    verbose=2,
    process=Process.hierarchical,     #2
    manager_llm=ChatOpenAI(               #3
        temperature=0, model="gpt-4"      #3
    ),    #4
)         #4
```

#1 Imports the LLM connector from LangChain
#2 You must set a crew manager when selecting hierarchical processing.
#3 Sets the crew manager to be the LLM connector
#4 You must set a crew manager when selecting hierarchical processing.

Run this file in VS Code (F5). When prompted, enter a game you want to create. Try using the same game you tried with AutoGen; the snake game is also a good baseline example. Observe the agents work through the code and review it repeatedly for problems.

After you run the file, you can also jump on AgentOps to review the cost of this run. Chances are, it will cost over double what it would have without the agent manager. The output will also likely not be significantly better. This is the trap of building agent systems without understanding how quickly things can spiral.

An example of this spiral that often happens when agents continually iterate over the same actions is frequently repeating tasks. You can view this problem in AgentOps, as shown in figure 4.13, by viewing the Repeat Thoughts plot.